

Domain-Driven Design · Symfony 8.1 · PHP 8.4

Проект «Catalog»

Как устроен бэкенд каталога товаров и зачем нужен каждый его слой.
Объяснение для тех, кто не знаком с DDD и Symfony.

Учебный разбор архитектуры · бэкенд `higload_app`

Один ограниченный контекст (bounded context): **Catalog**

Два HTTP-эндпоинта: создать товар и получить товар по id

Содержание

1. Что это за проект и что он умеет
2. Что такое Symfony (и зачем фреймворк)
3. Что такое DDD и какую проблему он решает
4. Четыре слоя архитектуры
5. Главные понятия DDD на примере этого проекта
6. CQRS: команды и запросы
7. Механизмы Symfony, которые всё связывают
8. Сквозной сценарий №1: создание товара (POST)
9. Сквозной сценарий №2: чтение товара (GET)
10. Карта файлов проекта
11. Зачем всё это: выгоды подхода
12. Краткий словарь терминов

1. Что это за проект и что он умеет

Это **серверная часть (бэкенд)** интернет-магазина — а точнее, его маленький кусочек, отвечающий за **каталог товаров**. Сейчас он умеет ровно две вещи, доступные по сети как HTTP-запросы (то, что браузер или мобильное приложение отправляет на сервер):

- **Создать товар** — `POST /api/catalog/products`. Клиент присылает артикул, название, цену, остаток на складе — сервер сохраняет новый товар в базу данных.
- **Получить товар по идентификатору** — `GET /api/catalog/products/{id}`. Клиент присылает `id` — сервер возвращает данные товара в формате JSON.

Аналогия. Представьте склад. POST — это «оформить новый товар и положить его на полку». GET — это «принеси мне карточку товара с такой-то биркой». Весь проект — это правила и сотрудники, которые делают эти две операции аккуратно и без ошибок.

Технологии: язык **PHP 8.4**, фреймворк **Symfony 8.1**, база данных **PostgreSQL** (через библиотеку Doctrine), быстрый кэш **Redis**. Дальше разберём, что значит каждое слово и зачем оно тут.

2. Что такое Symfony (и зачем нужен фреймворк)

Фреймворк — это готовый «каркас» приложения. Без него вам пришлось бы самим писать кучу скучного кода: как принять HTTP-запрос, как разобрать адрес (URL), как достать данные из базы, как вернуть ответ. Фреймворк всё это уже умеет — вы пишете только свою бизнес-логику.

Symfony — один из двух самых популярных PHP-фреймворков. Из того, что он даёт нашему проекту, важны четыре вещи:

Механизм Symfony	Что делает простыми словами
Роутинг (Routing)	Сопоставляет адрес запроса (<code>/api/catalog/products</code>) с конкретным методом в коде, который его обработает.
Контейнер / DI	«Сборщик» объектов. Сам создаёт нужные классы и передаёт их друг другу. Подробнее — в разделе 7.
Messenger	«Почта» внутри приложения: пересылает сообщения (команды, запросы, события) от отправителя к обработчику.
Doctrine	Переводчик между объектами PHP и таблицами базы данных (ORM). Вы работаете с объектом <code>Product</code> , а он сам пишет SQL.

Главная мысль

Symfony — это про *технику* (HTTP, база, очереди). DDD (следующий раздел) — это про то, как *организовать бизнес-логику*, чтобы она не превратилась в кашу. Эти две вещи дополняют друг друга.

3. Что такое DDD и какую проблему он решает

DDD (Domain-Driven Design, «проектирование на основе предметной области») — это подход к организации кода. Главная идея простая: **в центре приложения должны быть бизнес-правила, а не база данных и не фреймворк.**

Проблема, которую он решает

В типичном «быстром» приложении бизнес-правила размазаны повсюду: немного в контроллере, немного в SQL-запросе, немного в шаблоне. Пока проект маленький — нормально. Когда он растёт, такой код становится трудно менять: одно изменение ломает три других места, а понять «где же правило про отрицательный остаток» невозможно.

Решение DDD

DDD говорит: выдели **«предметную область» (Domain)** — сердце с чистыми бизнес-правилами — и не пускай туда технические детали. База данных, HTTP, кэш — это «*périphérie*», они снаружи и могут меняться, не задевая ядро.

Аналогия с рестораном. Domain — это рецепты и стандарты блюд (что считается правильным борщом). Infrastructure — это плита, холодильник, поставщик продуктов (их можно заменить, рецепт не изменится). UI — это официант, принимающий заказ. DDD требует: рецепт не должен зависеть от того, какой марки у вас холодильник.

Ключевой принцип: зависимости направлены внутрь

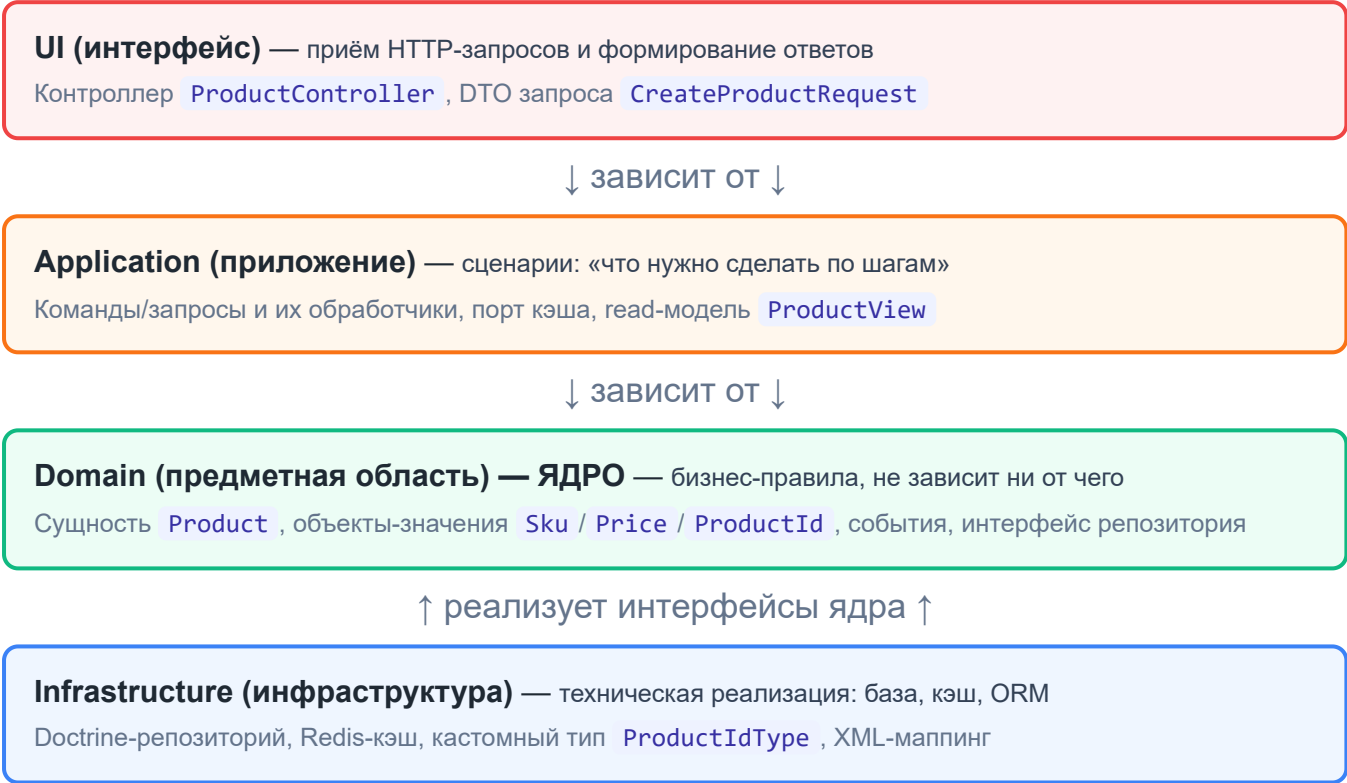
Внешние слои (UI, Infrastructure) знают о ядре (Domain) и зависят от него. Но **ядро не знает о них ничего**

. **Product** не подозревает, что существует PostgreSQL или HTTP. Благодаря этому ядро легко тестировать и оно не ломается при смене технологий.

4. Четыре слоя архитектуры

Проект разложен по папкам, и каждая папка — это «слой» с чёткой ответственностью.

Посмотрите на путь к файлам: `src/Catalog/Domain/...` , `.../Application/...` , `.../Infrastructure/...` , `.../UI/...` .



5. Главные понятия DDD на примере этого проекта

5.1. Ограниченный контекст (Bounded Context)

Большую систему делят на самостоятельные части — контексты. У нас один контекст — **Catalog** (каталог). В большом магазине были бы ещё «Корзина», «Оплата», «Доставка». Каждый контекст — свой мир со своими правилами и своим словарём. Это видно по пути `src/Catalog/...` — всё лежит внутри одного контекста.

5.2. Сущность и Агрегат (Entity / Aggregate): `Product`

Сущность — объект, у которого есть уникальный идентификатор и своя «жизнь» во времени (его можно менять, но это всё тот же объект). Наш `Product` — это сущность и одновременно **агрегат**: главный объект, который отвечает за свою целостность.

Важно: правила живут *внутри* объекта. Нельзя положить отрицательный остаток — объект сам это запретит:

```
public function updateStock(int $newStock): void
{
    if ($newStock < 0) throw new \DomainException('Stock cannot be negative');
    $this->stock = $newStock;
    $this->updatedAt = new \DateTimeImmutable();
    $this->recordUpdated(); // запомнить, что товар изменился
}
```

Почему это хорошо

Невозможно «случайно» создать неправильный товар из другого места программы. Правило в одном месте — в самом объекте. Это и есть «защита инвариантов».

5.3. Объект-значение (Value Object): `Sku`, `Price`, `ProductId`

Объект-значение — это маленький неизменяемый объект, который описывается только своим значением и не имеет собственного «id». Деньги, артикул, идентификатор — отличные кандидаты.

Зачем заворачивать цену в класс `Price`, а не хранить просто число? Чтобы **правила и смысл** были рядом со значением:

```
final class Price
{
    public function __construct(private int $amount, private string $currency = 'KZT')
    {
        if ($amount < 0) throw new \InvalidArgumentException('Price cannot be negative');
    }
}
```

```
// два объекта-значения равны, если равны их значения
public function equals(self $other): bool
{
    return $this->amount === $other->amount && $this->currency === $other->currency;
}
}
```

Аналогия. Купюра в 100 тенге равноценна любой другой купюре в 100 тенге — нам не важен её серийный номер. Это «объект-значение». А вот сотрудник Иван — это «сущность»: даже два полных тёзки — разные люди.

Артикул `Sku` хранит правило формата прямо в себе, в виде константы (её же переиспользует слой UI для проверки ввода — правило не дублируется):

```
final class Sku
{
    public const PATTERN = '/^[A-Z0-9]{8,20}$/'; // 8-20 заглавных букв и цифр
}
```

5.4. Доменные события (Domain Events): `ProductCreatedEvent`, `ProductUpdatedEvent`

Доменное событие — это факт «что-то важное произошло в бизнесе»: *товар создан*, *товар изменён*. Агрегат сам «записывает» такие факты, когда меняется. Позже их можно разослать другим частям системы (поиск, аналитика, уведомления) — и им не нужно для этого лезть внутрь нашего кода.

```
// внутри Product: накапливаем события, потом их «забирают»
public function pullDomainEvents(): array
{
    $events = $this->domainEvents;
    $this->domainEvents = []; // очищаем, чтобы не разослать дважды
    return $events;
}
```

Тонкость DDD, которую мы исправили

Раньше событие собиралось в обработчике (снаружи объекта). Это неправильно: «знание о том, что товар создан» — принадлежит самому товару. Теперь `Product` сам записывает событие в конструкторе, а обработчик лишь забирает и рассылает.

5.5. Репозиторий (Repository) и «порт»

Репозиторий — это «хранилище объектов»: умеет сохранить товар и найти его по id. Хитрость DDD: ядро объявляет *только интерфейс* (что должно уметь хранилище), а *как именно* — пишет Infrastructure.


```
// Domain: контракт (порт). Никакого SQL — только намерение.  
interface ProductRepositoryInterface  
{  
    public function save(Product $product): void;  
    public function findById(ProductId $id): ?Product;  
}
```

```
// Infrastructure: реализация через Doctrine (здесь уже база)  
class ProductRepositoryDoctrine implements ProductRepositoryInterface  
{  
    public function save(Product $product): void  
    {  
        $this->em->persist($product);  
        $this->em->flush(); // здесь Doctrine пишет SQL INSERT  
    }  
}
```

Зачем такая «двойственность»

Это называется «инверсия зависимостей». Завтра вы смените PostgreSQL на MongoDB — меняете только класс в Infrastructure, а всё ядро и сценарии останутся нетронутыми, потому что они зависят от *интерфейса*, а не от базы.

6. CQRS: команды и запросы

CQRS (Command Query Responsibility Segregation) — «разделение ответственности команд и запросов». Идея в одном предложении: **операции, которые меняют данные, и операции, которые только читают данные, — это разные миры, и обрабатывать их надо по-разному.**

	Команда (Command)	Запрос (Query)
Цель	Изменить состояние («создай товар»)	Прочитать состояние («дай товар»)
Возвращает	Ничего (или только подтверждение)	Данные для показа
В проекте	CreateProductCommand → CreateProductHandler	GetProductQuery → GetProductHandler
Идёт через шину	command.bus	query.bus

Команда — это просто «конверт с данными» о намерении. Сама она ничего не делает; её исполняет **обработчик (handler)**:

```
class CreateProductCommand // просто данные намерения
{
    public function __construct(
        public readonly Sku $sku,
        public readonly string $name,
        public readonly Price $price,
        public readonly int $stock,
        public readonly ?string $description
    ) {}
}
```

Зачем разделять

Чтение часто можно ускорить (кэш, упрощённые модели данных), а запись — требует строгих проверок. Разделив их, мы оптимизируем каждую сторону отдельно. В нашем проекте чтение использует кэш Redis и лёгкую модель `ProductView`, а запись идёт через полноценный агрегат `Product` с проверками.

7. Механизмы Symfony, которые всё связывают

7.1. Контейнер и внедрение зависимостей (Dependency Injection)

Объекту `CreateProductHandler` нужны репозиторий и шина событий. Он их **не создаёт сам** — он просто заявляет в конструкторе «мне нужно вот это», а Symfony-контейнер при запуске сам создаёт и «вкладывает» нужные объекты. Это и есть **внедрение зависимостей**.

Аналогия. Вы не строите автомобиль, чтобы доехать до работы, — вам его «подают». Классу подают всё, что ему нужно, готовым.

В файле `config/services.yaml` мы говорим контейнеру, какую *реализацию* подставлять под *интерфейс*:

```
# «Когда кому-то нужен интерфейс репозитория — дай Doctrine-реализацию»
App\Catalog\Domain\Repository\ProductRepositoryInterface:
    '@App\Catalog\Infrastructure\Doctrine\ProductRepositoryDoctrine'
```

7.2. Messenger и три шины

Шина (bus) — это «почтовая служба»: вы бросаете в неё сообщение, она находит нужный обработчик и доставляет. У нас три шины под три типа сообщений (`config/packages/messenger.yaml`):

Шина	Для чего
<code>command.bus</code>	Команды на изменение (создать товар)
<code>query.bus</code>	Запросы на чтение (получить товар)
<code>event.bus</code>	Доменные события (товар создан/изменён) — рассылаются асинхронно

7.3. Doctrine ORM: объекты ↔ таблицы

ORM (Object-Relational Mapping) избавляет от ручного написания SQL. Вы работаете с объектом `Product`, а Doctrine знает, в какую таблицу и колонки его положить. «Карта» соответствия задана XML-файлами в `Infrastructure/Doctrine/Mapping`.

Объекты-значения `Sku` и `Price` хранятся не в отдельных таблицах, а «встраиваются» в строку товара (это называется *embeddable*): цена превращается в две колонки — `price_amount` и `price_currency`.

А чтобы наш `ProductId` (объект) корректно превращался в строку UUID для базы и обратно, написан **кастомный тип** `ProductIdType`:

```
// объект ProductId → строка для базы
public function convertToDatabaseValue($value, $platform): ?string
{
    return $value instanceof ProductId ? $value->toString() : null;
}
```

7.4. Кэш Redis за «портом»

Кэш — быстрая память для часто запрашиваемых данных, чтобы не дёргать базу каждый раз. **Redis** — популярное хранилище для этого. Чтобы слой Application не зависел от Redis напрямую, мы спрятали кэш за интерфейс-порт `ProductCacheInterface`; реализация `ProductCacheRedis` живёт в Infrastructure. Кэш хранит лёгкую read-модель `ProductView` на 5 минут.

7.5. Контроллер и проверка входных данных

Контроллер — «дверь» приложения. Symfony сам превращает входной JSON в объект `CreateProductRequest` и проверяет его (атрибут `#[MapRequestPayload]`). Если данные кривые — клиент получит ошибку ещё до того, как они дойдут до бизнес-логики.

```
#[Route('/api/catalog/products', methods: ['POST'])]
public function create(#[MapRequestPayload] CreateProductRequest $request): JsonResponse
```

8. Сквозной сценарий №1: создание товара (POST)

Проследим, что происходит, когда приходит `POST /api/catalog/products` с JSON товара. Обратите внимание, как запрос идёт **сверху вниз по слоям**.

- 1 **UI.** Symfony по адресу находит метод `ProductController::create`. JSON автоматически превращается в `CreateProductRequest` и проверяется (артикул, длина имени, цена > 0 , остаток ≥ 0).
- 2 **UI → Application.** Контроллер собирает из проверенных данных `CreateProductCommand` (упаковывая `Sku` и `Price` как объекты-значения) и бросает её в `command.bus`.
- 3 **Application.** Шина доставляет команду в `CreateProductHandler`. Тот создаёт **новый агрегат** `new Product(...)` — и в этот момент сам товар проверяет правила и записывает событие «товар создан».
- 4 **Application → Domain/Infrastructure.** Обработчик вызывает `repository->save($product)`. Вызов идёт на *интерфейс*, а выполняет его Doctrine-репозиторий — он пишет строку в PostgreSQL.
- 5 **События.** Обработчик забирает у товара события (`pullDomainEvents()`) и рассылает их в `event.bus` — чтобы потом сработали поиск, уведомления и т.п.
- 6 **Ответ.** Контроллер возвращает JSON `{"status":"Product created"}` и HTTP-код **201 Created**.

```
// Шаги 2-3 в коде обработчика
public function __invoke(CreateProductCommand $command): void
{
    $product = new Product($command->sku, $command->name, $command->price,
                           $command->stock, $command->description);
    $this->repository->save($product);

    foreach ($product->pullDomainEvents() as $event) {
        $this->eventBus->dispatch($event); // разослать «товар создан»
    }
}
```

9. Сквозной сценарий №2: чтение товара (GET)

Теперь `GET /api/catalog/products/{id}`. Здесь в игру вступает кэш.

- 1 **UI.** `ProductController::get` пытается превратить строку `{id}` в объект `ProductId`. Если `id` невалидный — сразу ответ **400 Bad Request** (а не падение сервера).
- 2 **UI → Application.** Контроллер бросает `GetProductQuery` в `query.bus`; шина возвращает «конверт» (Envelope), из которого достаётся результат обработчика.
- 3 **Application + кэш.** `GetProductHandler` спрашивает кэш: «есть товар с таким `id`?». Если есть — отдаёт мгновенно. Если нет — выполняется «добытчик»: идём в репозиторий за товаром.
- 4 **Infrastructure.** Репозиторий достаёт `Product` из PostgreSQL. Обработчик превращает тяжёлый агрегат в лёгкую read-модель `ProductView` (только нужные для показа поля) и кладёт её в кэш на 5 минут.
- 5 **Ответ.** Если товар найден — `ProductView` превращается в JSON (код **200 OK**). Если нет — **404 Not Found**.

```
public function __invoke(GetProductQuery $query): ?ProductView
{
    return $this->cache->get($query->id, function () use ($query): ?ProductView {
        $product = $this->repository->findById($query->id); // промах кэша → база
        return $product ? ProductView::fromEntity($product) : null;
    });
}
```

Почему отдаём `ProductView`, а не `Product`

Тяжёлый агрегат с правилами незачем (и небезопасно) складывать в кэш и слать наружу. Лёгкая «витрина» `ProductView` содержит только то, что нужно показать, и легко превращается в JSON.

10. Карта файлов проекта

Где что лежит и за что отвечает (всё внутри `src/Catalog/`):

Файл	Слой	Назначение
<code>Domain/Entity/Product.php</code>	Domain	Агрегат-товар: правила, события
<code>Domain/ValueObject/Sku.php</code>	Domain	Артикул + правило формата
<code>Domain/ValueObject/Price.php</code>	Domain	Цена + валюта, запрет отрицательной
<code>Domain/ValueObject/ProductId.php</code>	Domain	Идентификатор товара (UUID)
<code>Domain/Event/ProductCreatedEvent.php</code>	Domain	Факт «товар создан»
<code>Domain/Event/ProductUpdatedEvent.php</code>	Domain	Факт «товар изменён»
<code>Domain/Repository/ProductRepositoryInterface.php</code>	Domain	Контракт хранилища (порт)
<code>Application/Command/CreateProductCommand.php</code>	Application	Намерение «создать товар»
<code>Application/Command/CreateProductHandler.php</code>	Application	Сценарий создания
<code>Application/Query/GetProductQuery.php</code>	Application	Намерение «получить товар»
<code>Application/Query/GetProductHandler.php</code>	Application	Сценарий чтения (с кэшем)
<code>Application/Query/ProductView.php</code>	Application	Лёгкая read-модель для показа
<code>Application/Port/ProductCacheInterface.php</code>	Application	Контракт кэша (порт)
<code>Infrastructure/Doctrine/ProductRepositoryDoctrine.php</code>	Infrastructure	Хранилище через PostgreSQL
<code>Infrastructure/Doctrine/Type/ProductIdType.php</code>	Infrastructure	Перевод ProductId ↔ строка БД
<code>Infrastructure/Cache/ProductCacheRedis.php</code>	Infrastructure	Кэш через Redis

UI/Http/ProductController.php	UI	HTTP-эндпоинты POST и GET
UI/Http/Dto/CreateProductRequest.php	UI	Проверка входного JSON

11. Зачем всё это: выгоды подхода

Слоёв и абстракций больше, чем в «простом» коде — и это окупается, когда проект растёт:

- **Бизнес-правила в одном месте.** Хотите узнать все правила про товар — открываете `Product`, а не ищете по всему проекту.
- **Легко тестировать.** Ядро не зависит от базы и сети, поэтому правила проверяются быстрыми unit-тестами без запуска PostgreSQL/Redis. В проекте уже есть такие тесты (12 шт.) — они проверяют события агрегата и объекты-значения за доли секунды.
- **Технологии заменяемы.** Сменить базу или кэш = поменять класс в Infrastructure. Ядро и сценарии не трогаются.
- **Чтение и запись масштабируются отдельно** (благодаря CQRS): читаем — из кэша и быстрых моделей, пишем — со строгими проверками.
- **Понятный словарь.** Код говорит на языке бизнеса (Product, Sku, Price, «товар создан»), а не на языке таблиц.

Честно о минусах

DDD — это «инвестиция». Для крошечного скрипта он избыточен: больше файлов, больше понятий. Он оправдан там, где логика сложная и проект живёт долго (как раз случай растущего магазина — «highload»-приложения, судя по названию проекта).

12. Краткий словарь терминов

Термин	Простыми словами
DDD	Подход: в центре кода — бизнес-правила, а не база/фреймворк
Bounded Context	Самостоятельная часть системы со своим словарём (у нас — Catalog)
Domain	Ядро с бизнес-правилами; ни от чего не зависит
Application	Сценарии «что сделать по шагам»; дирижёр
Infrastructure	Технические детали: база, кэш, ORM
UI	Приём запросов и выдача ответов (HTTP/JSON)
Entity / Сущность	Объект с id и историей изменений (Product)
Aggregate / Агрегат	Главная сущность, отвечающая за свою целостность
Value Object	Неизменяемый объект, описываемый только значением (Price, Sku)
Domain Event	Факт «что-то важное произошло» (товар создан)
Repository	Хранилище объектов: сохранить / найти
Port (порт)	Интерфейс в ядре; реализацию даёт Infrastructure
CQRS	Разделение операций записи (Command) и чтения (Query)
DI / Контейнер	Symfony сам создаёт объекты и «подаёт» их зависимостям
Messenger / Bus	Внутренняя «почта»: доставляет сообщения обработчикам
ORM / Doctrine	Переводчик «объекты ↔ таблицы базы данных»
DTO	Простой объект-«конверт» для передачи данных (запрос, ProductView)

Документ сгенерирован как учебное объяснение архитектуры проекта `higload_app/backend` (контекст Catalog).